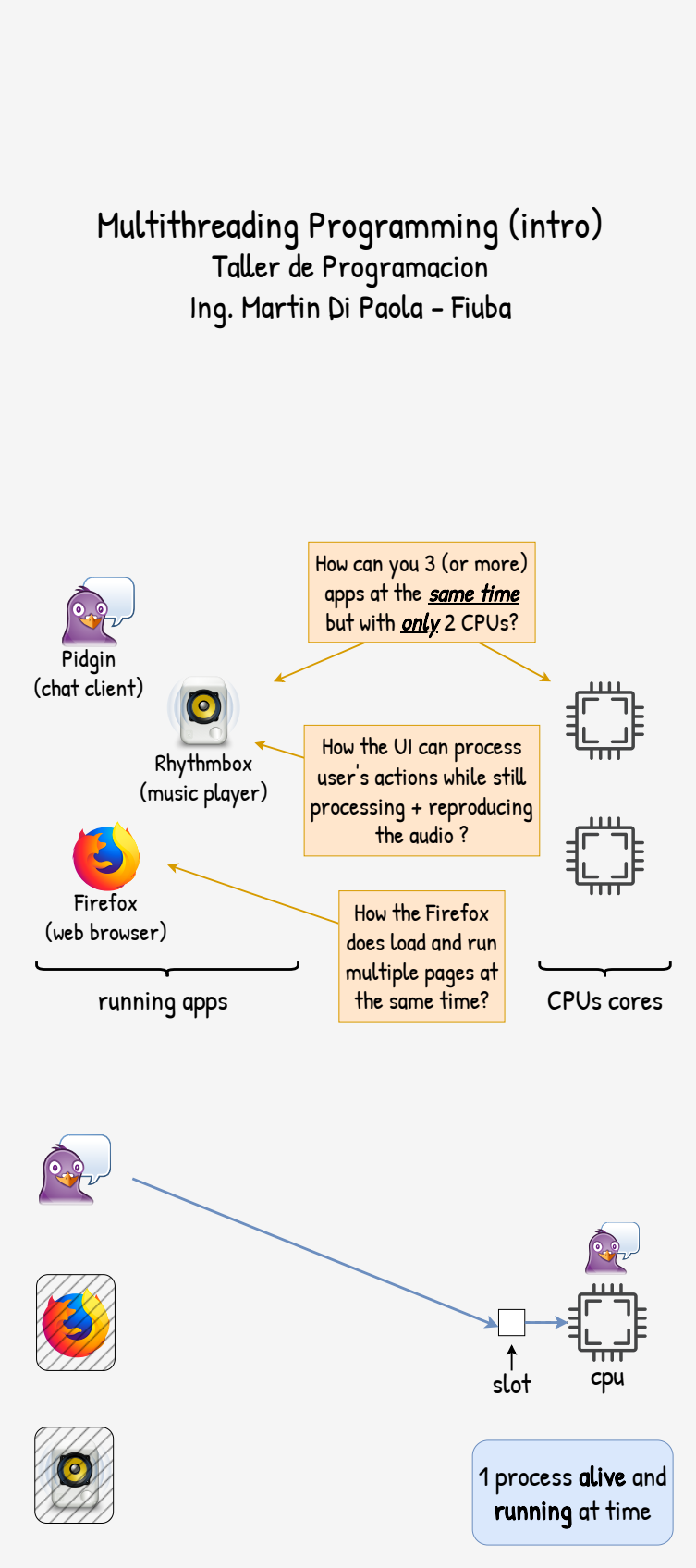



Si bien hardware moderno tiene N CPU cores (2, 4, 16, ...). Como es posible que se puedan correr M > N procesos? Como es posible q un proceso pueda estar haciendo varias cosas a la vez?

En OS antiguos (hasta principios de los 90s) solo se soportaba un único proceso en ejecución.

Cuando el user quería correr un programa, el kernel le daba control completo a la aplicación.

No solo es poco friendly para el user correr una solo cosa a la vez sino tambien un problema de seguridad. Si el proceso se colgaba (tipo un loop infinito), acaparaba toda la CPU y nadie, ni siquiera el kernel podian hacer algo.

Es tech vieja pero estos OS single task se siguen usando en embebidos.

```
void process() {
   show_interface();
   read_messages_from_others();
   format_messages();
   show_messages();
   wait_local_message();
   show_local_message();
   send_local_message()
}
```

Imaginate el código de Pidgin, un cliente de chat gráfico.

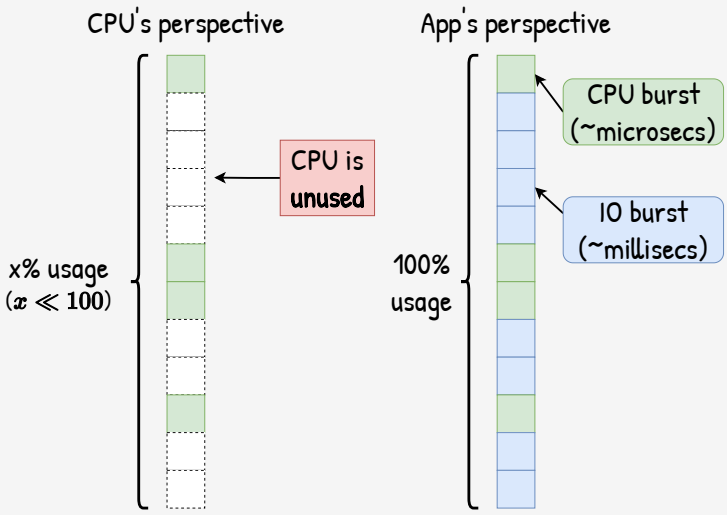
```
void process() {
   show_interface();
   read_messages_from_others();
   format_messages();
   show_messages();
   wait_local_message();
   show_local_message();
   send_local_message()
}
```

Algunas instrucciones hacen solo computo (renderizar, formatear) mientras que otras requieren interactuar con algun disposito (network, keyboard).

Asi un programa tiene momentos q hace uso de la CPU y momentos en los que solo esta esperando que alguna operación de IO termine.

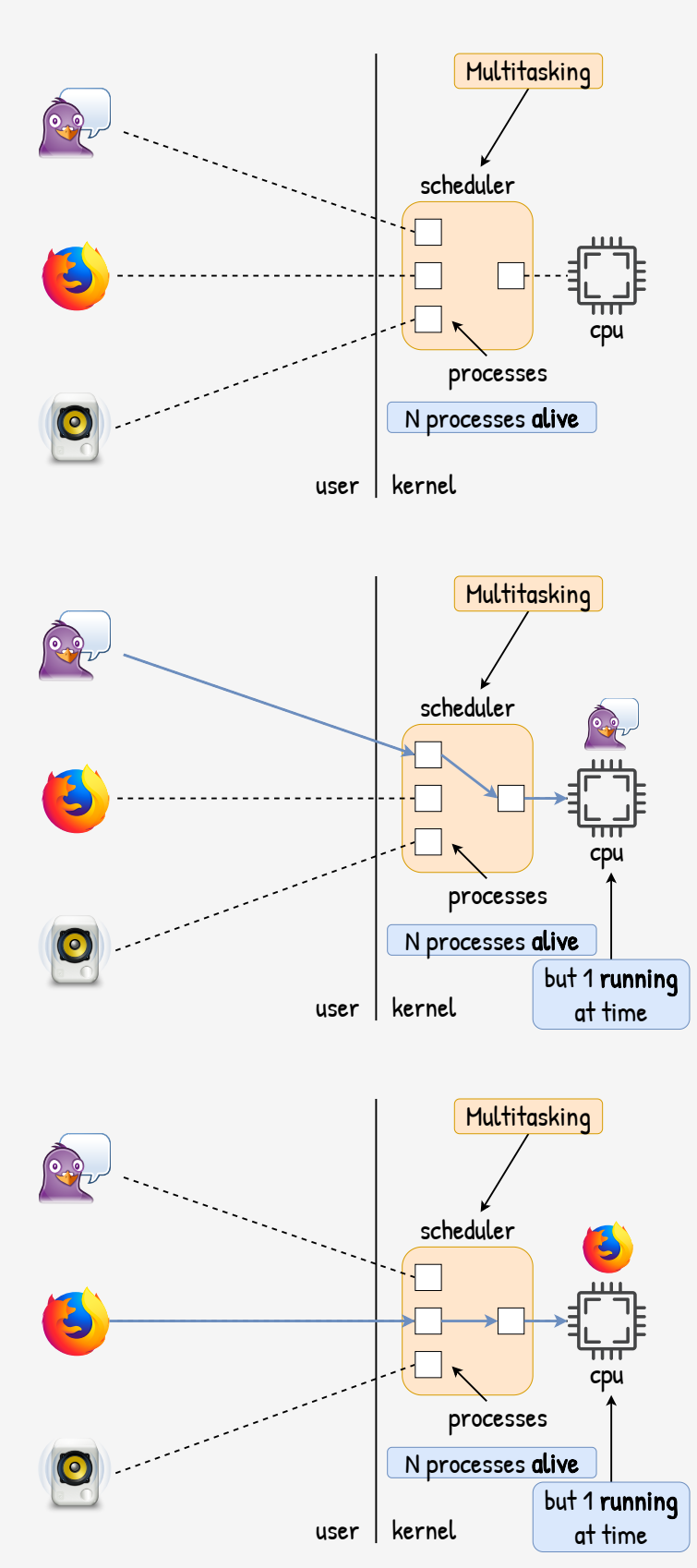
A los primeros se los llama CPU burst (ráfaga de CPU), a los segundos IO burst.

Notar hay diferencia de 2 o 3 ordenes de magnitud en los tiempos de ambas categorías: esperar IO es **mucho** más lento.




Desde la perspectiva de la app, ella esta todo tiempo o bien procesando datos (CPU) o esperando (IO). No tiene tiempos muertos (no tiene tiempos de espera innecesarios).

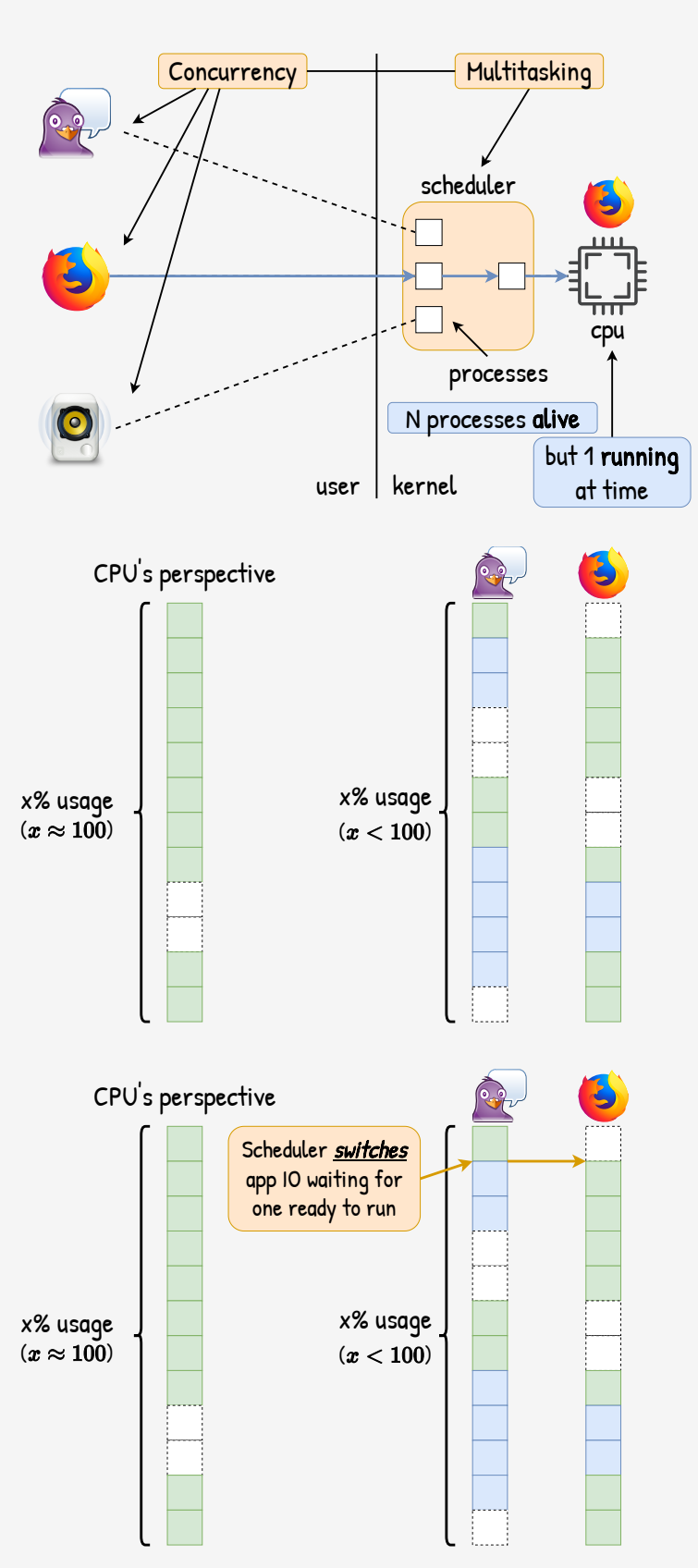
Desde la perspectiva de la CPU, este esquema es horrible. La gran mayoría del tiempo la CPU no esta siendo usada.




Entrados los 90s aparecieron los OS multitasking. El user podia correr varios programas a la vez aunque "a la vez" tiene un matiz.

Con solo un core, solo se puede ejecutar un proceso a la vez. Quien decide que procesos vivo pasa a o no a ser ejecutado en la CPU lo decide el scheduler del OS.

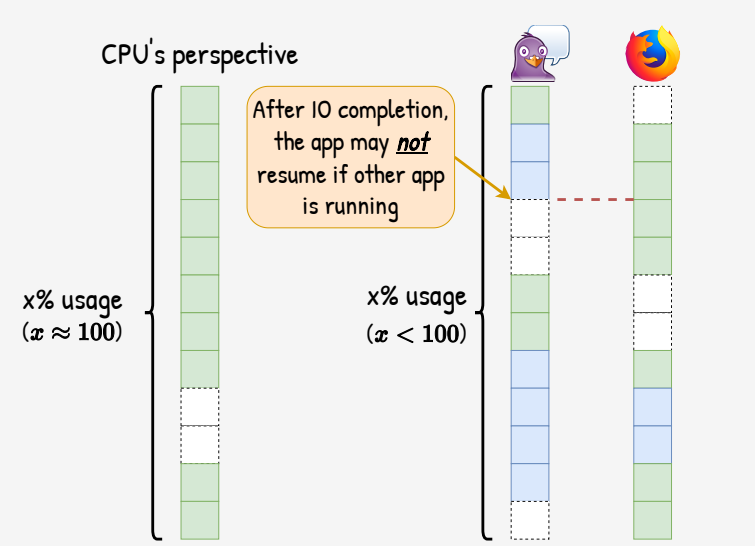
Aunque realmente los procesos no corren "a la vez" sino que se van turnando, tanto la performance como la usabilidad del sistema era mucho mejor que un esquema single-task.



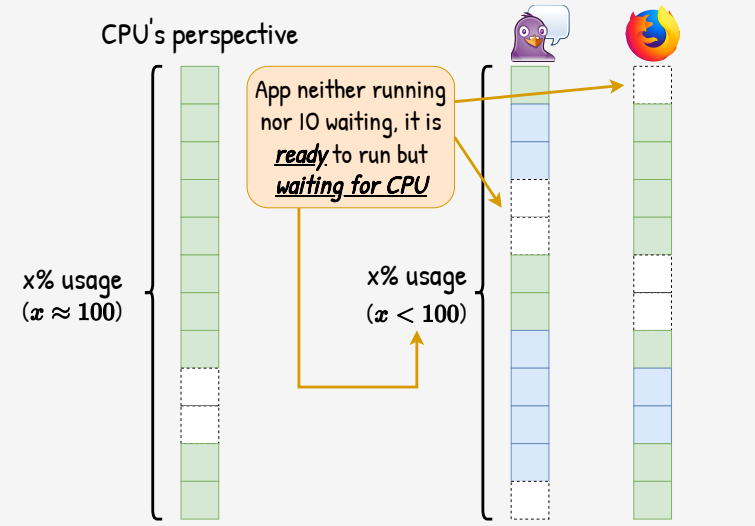

Este “a la vez” se lo llama concurrencia: los procesos corren concurrentemente.

.

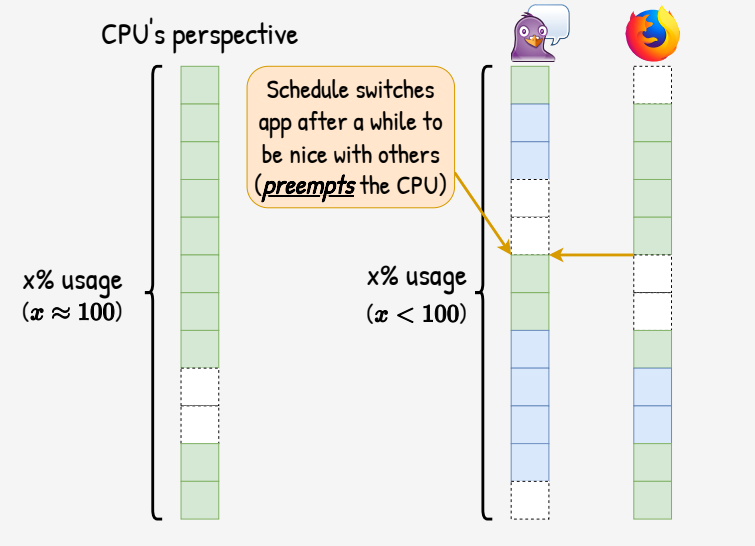
Cuando un proceso hace una operacion de IO, el kernel (el scheduler) sabe q no necesita la CPU en le corto plazo y hace un cambio, un context switch de un proceso a otro que este listo para correr.




Cuando la operacion de IO termina, el proceso queda listo para correr nuevamente pero no necesariamente el scheduler le va a asignar una CPU.

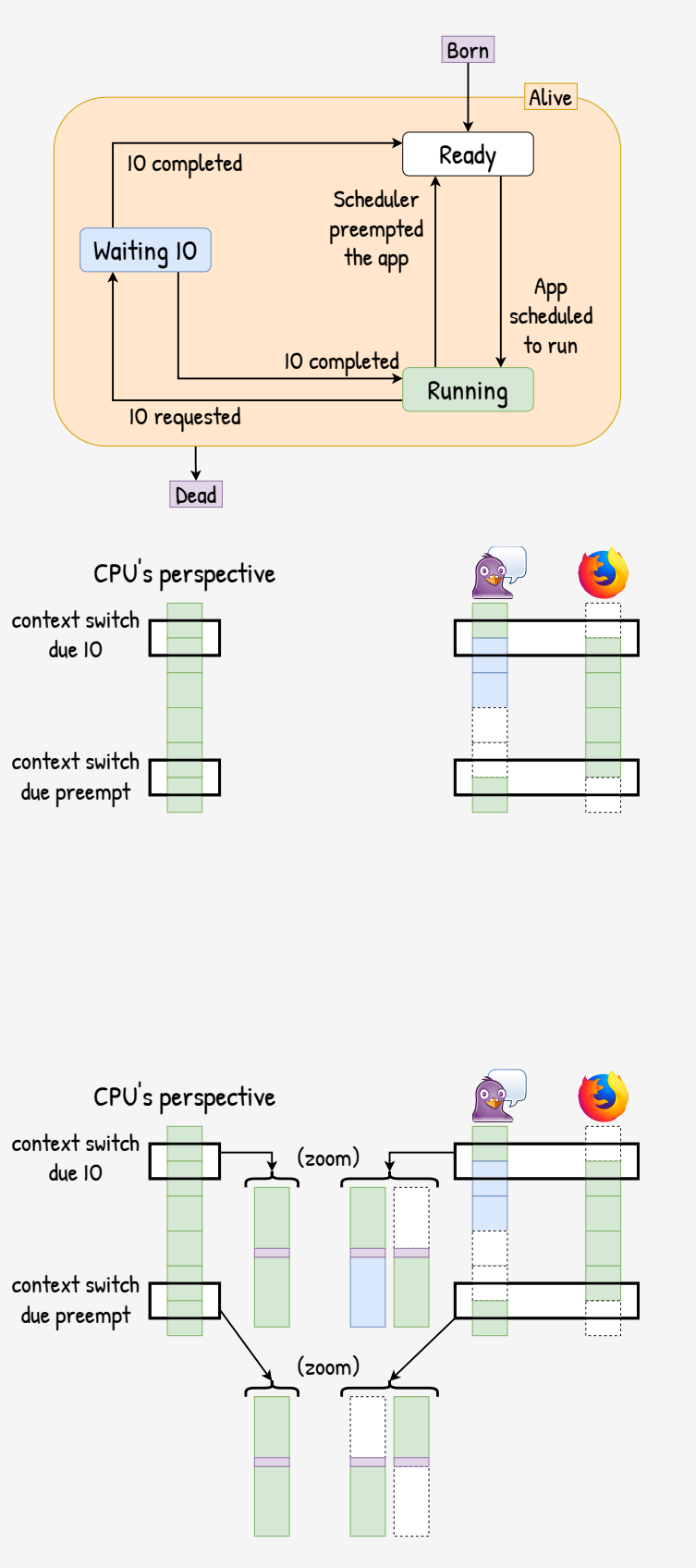



Desde la perspectiva del procesos, la concurrencia le introduce tiempos muertos y retrasos debido a quedar esperando a q la CPU quede libre.

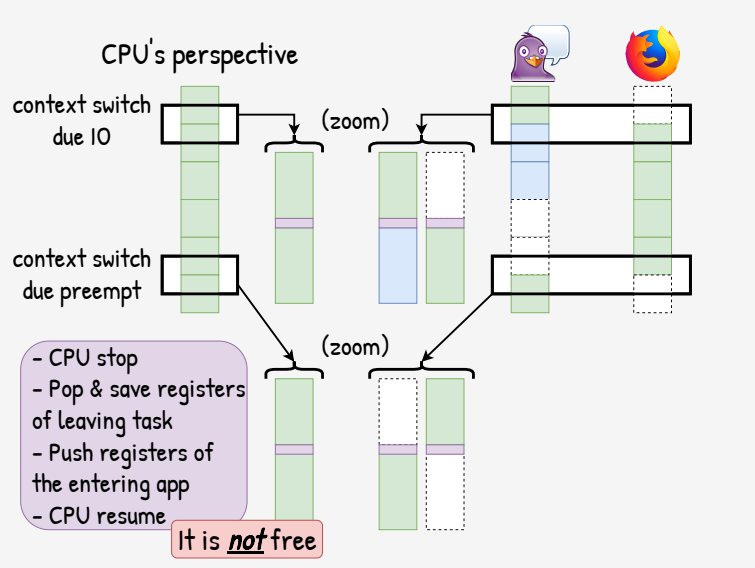



Los OS modernos ademas soportan preemption: el scheduler puede retirar (preempt) un proceso de la CPU y darle paso a otro aun sin el primero no hace una operacion de IO.

Esto le permite al kernel ser "justo" con otros procesos queriendo correr y a la vez protegerse de que ningun proceso quede en un loop infinito y cuelgue al sistema.



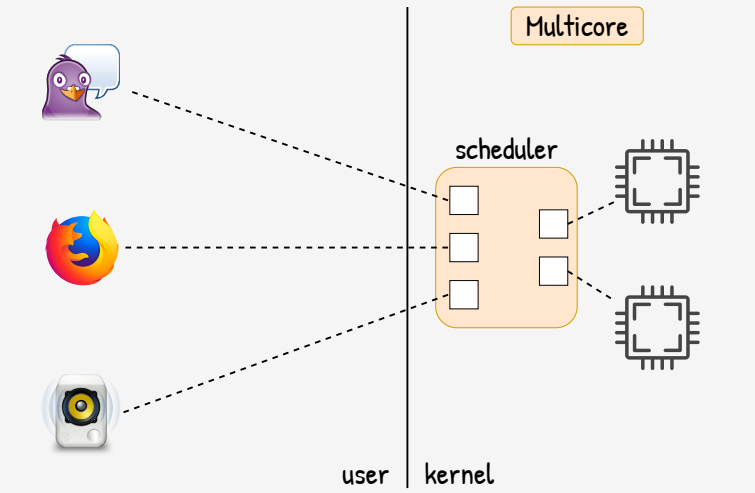

Cada vez q el scheduler hace un cambio de una task (sea por una operacion IO o por que simplemente una task ya estuvo demasiado tiempo en la CPU) hay un context switch.




El context switch no es instantaneo ni gratis. Hay un mini-procesamiento hecho por el kernel: la CPU hay q frenarla, los registros de la task saliente hay q sacarlos y guardarlos en memoria, los registros de la task entrante hay q sacarlos de la memoria y ponerlos en la CPU y finalmente reiniciar la ejecucion de la CPU.

Un context switch introduce delays y esto es esperable y hasta cierto punto inevitable.

Pero una alta frecuencia de context switches (por una alta frecuencia de IO o de concurrencia) termina degradando la performance del sistema.



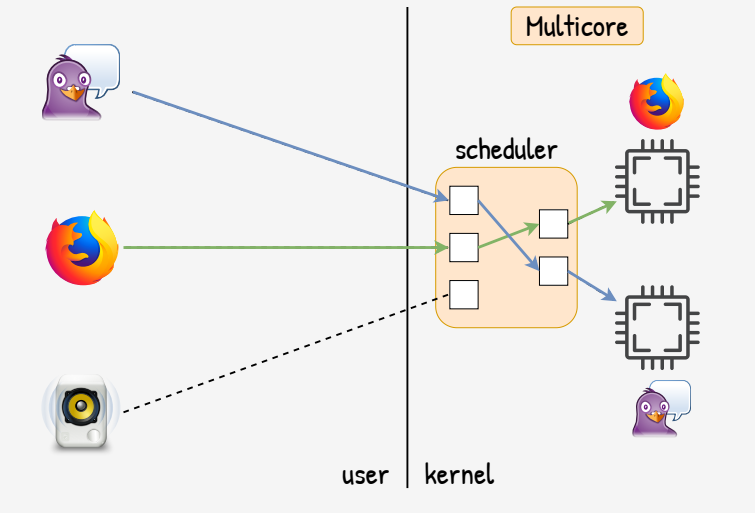

Y para inicios del 2000 los fabricantes de CPU estaban llegando al límite. Resulta que para hacer CPUs más rápidos hay que aumentar la frecuencia (el clock) y esto tiene 2 problemas:

- el calor generado es proporcional al cuadrado de la frecuencia
- a mayor frecuencia más chico deben ser las dimensiones de la CPU

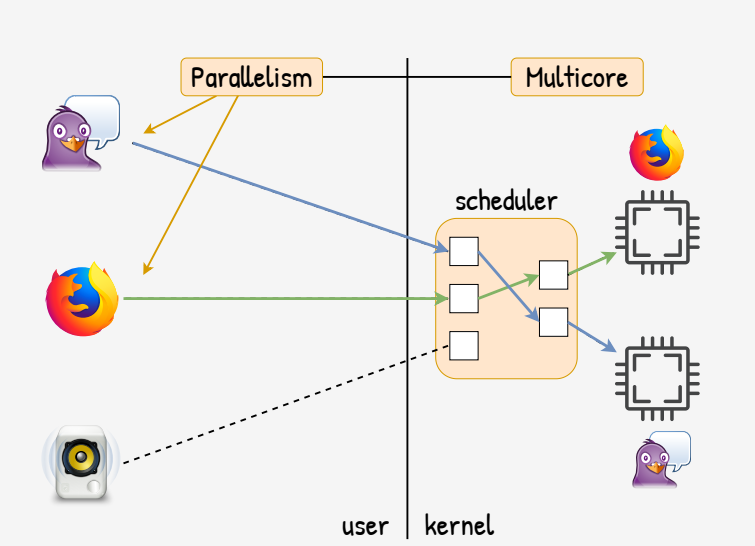
  para evitar distorciones de las señales electricas (pero al ser físicamente

  más chicos, es mucho más difícil disipar el calor!)

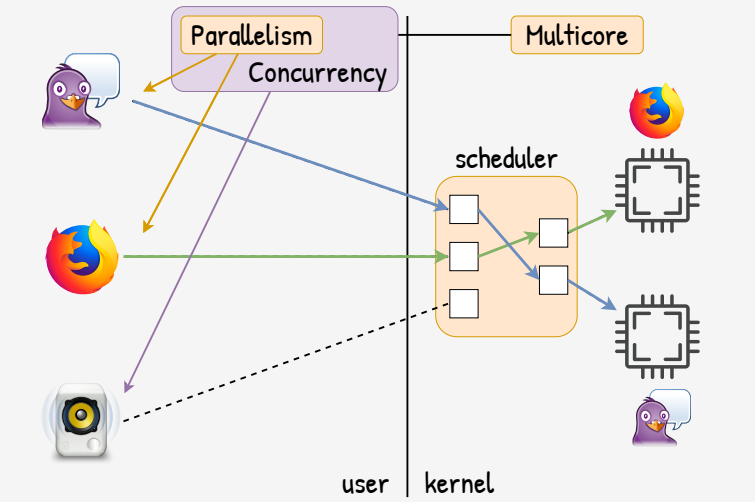
La CPU no podia ser mucho más rápida sin violar alguna ley de la física. Y asi aparecieron los multi-cores.




Al igual que antes el scheduler podia asignar procesos listos a una de las CPUs libres. Solo que ahora habia más de 1.



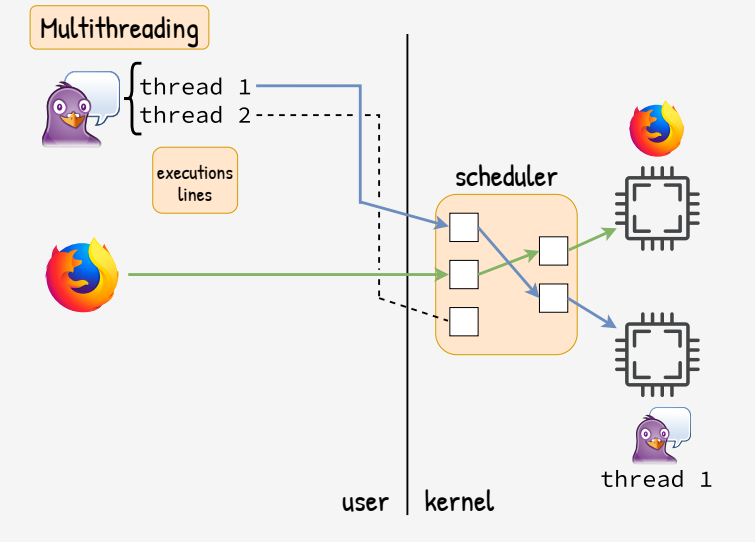

Ahora, los procesos realmente podian correr a la vez, en simultaneo, en paralelo.




Pero el paralelismo esta limitado por la cantidad de CPUs (cores) reales. El resto de los procesos corren concurrentemente.

Diferencia entre concurrencia y paralelismo? Ahora mismo estas leyendo y a la vez respirando, eso es paralelismo.

Si estuvieras comiendo algo, tambien estarias respirando pero no exactamente a la vez (o te ahogarias!). Eso es concurrencia.




Con multi-tasking se responde como varias apps pueden correr “a la vez”.

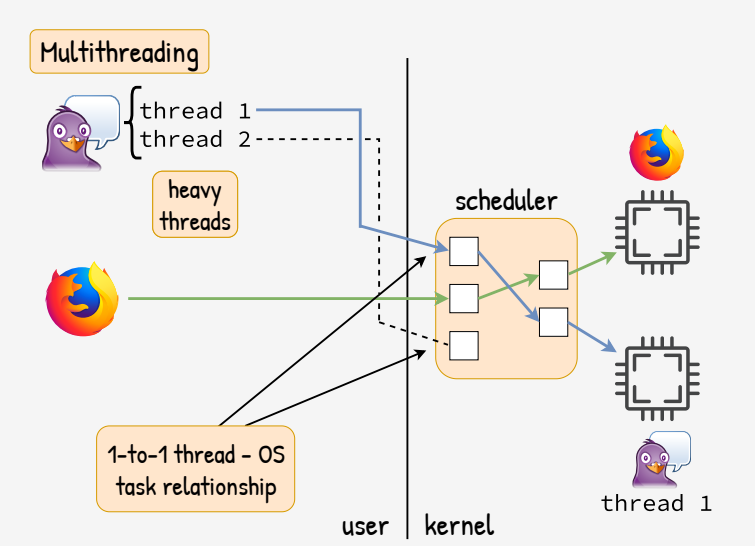
Pero como una misma app puede hacer múltiples cosas “a la vez”?

- Como puede una app gráfica responder a los clicks del mouse y a la vez reproducir música?
- Como puede un cliente de chat procesar lo q el user esta escribiendo y a la vez recibir y mostrar

  lo que otros le han enviado?

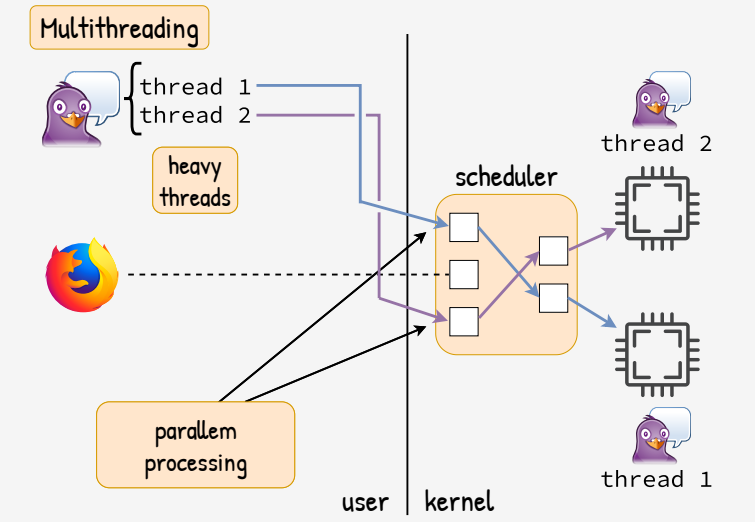
Un hilo (thread) es una sequencia o unidad de ejecucion con su propio stack y registros.

Todo proceso tiene al menos 1 thread.

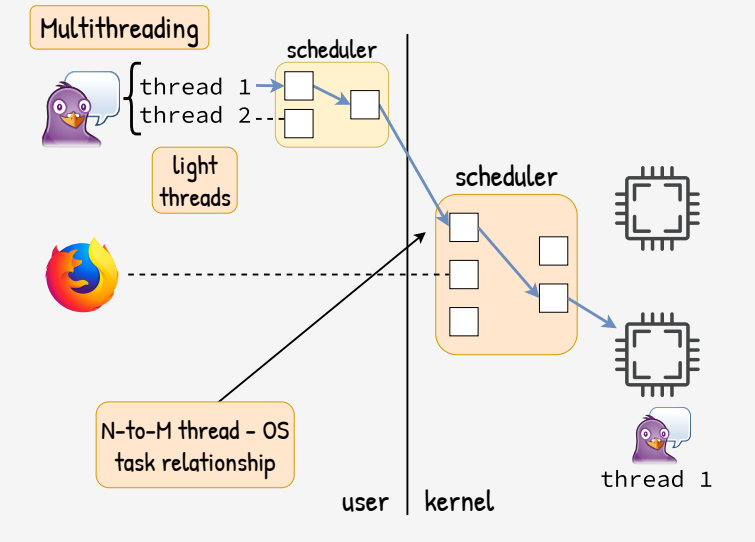



Los threads que tienen una correspondencia 1-a-1 con una task ejecutable del OS son heavy threads.

Son "pesados" (heavy) por que requieren recursos del OS para ser representados, como si fuesen procesos.




Como los threads son visibles por el scheduler del OS, este puede asignarlos a los CPU core haciendo que corran en paralelo (de otro modo, corren concurrentemente)



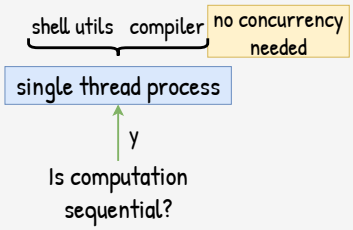

Pero los heavy threads demandan demasiados recursos y escalan mal.

Los hilos ligeros (light, green, o coroutines) son threads que no tiene una representacion en el kernel, consumen menos recursos y el switch de un thread a otro lo hace un scheduler que corre en userspace.

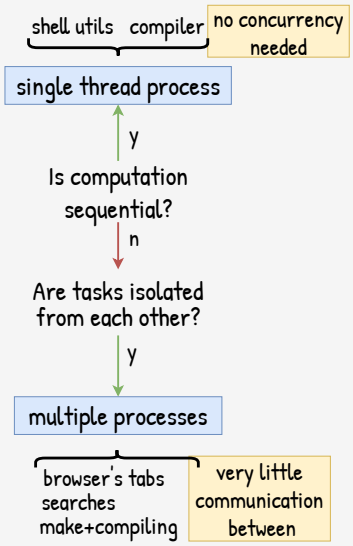
Es este scheduler quien determina cuantos heavy threads van a ser requeridos y por ende el nivel de paralelismo (en el diapositiva, hay un único heavy thread y por lo tanto los threads del Pidgin no pueden correr en paralelo)

# Takeaways

*Single thread process?*
*Multi-processes?*
*Single process, multi-threads?*
*Heavy or light threads?*



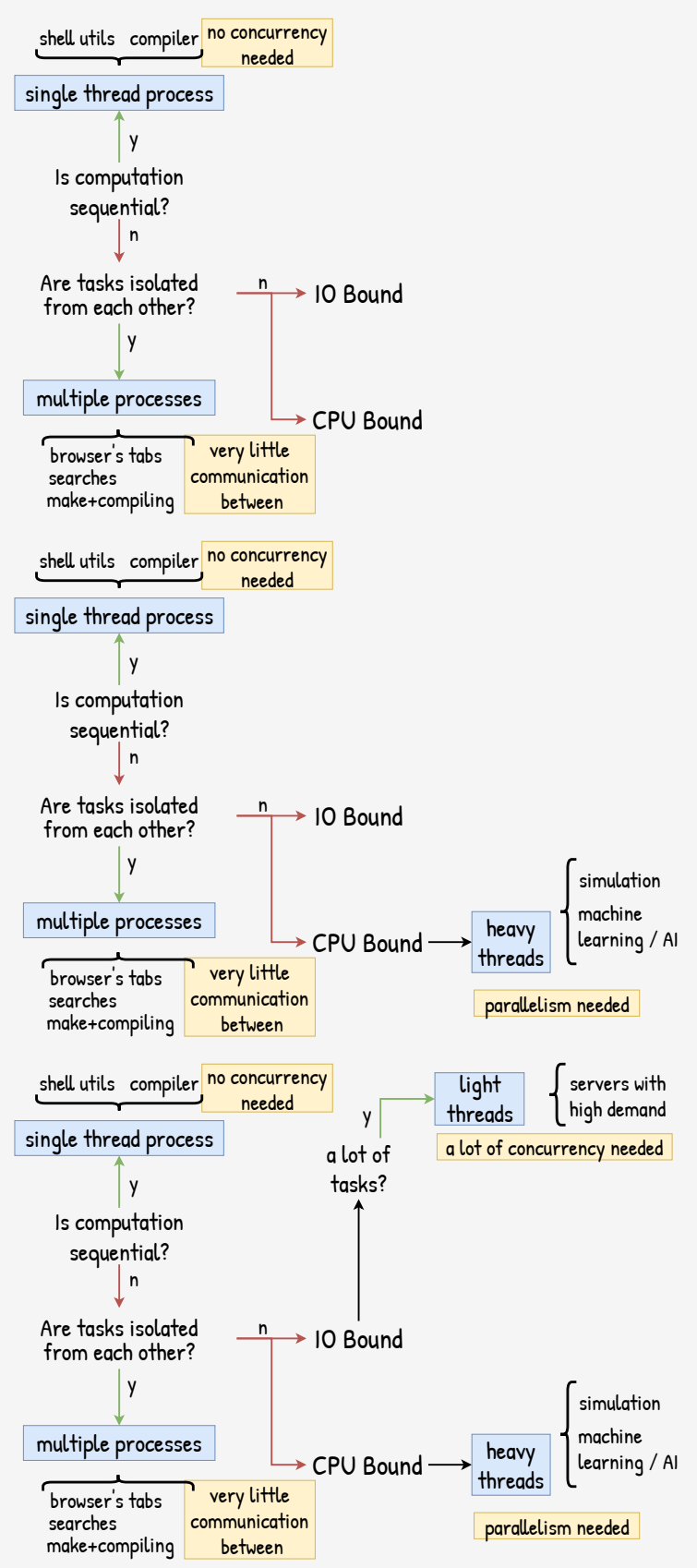

Ejemplos: `cat`, `ls`, `find`, la mayoría de las tools de un shell son single-thread.




Cuando se quiere paralelismo (o concurrencia) pero no hay necesidad de comunicación entre las partes (o es mínima), se pueden utilizar múltiples procesos.

Ejemplos:

- las tabs de un web browser, por seguridad, se asilan en procesos.
- al compilar un proyecto, `make` puede lanzar a varios compilers a la vez.




Con heavy threads se gana paralelismo. Tiene sentido cuando la tarea puede partirse en sub-tareas y todas son CPU bound.

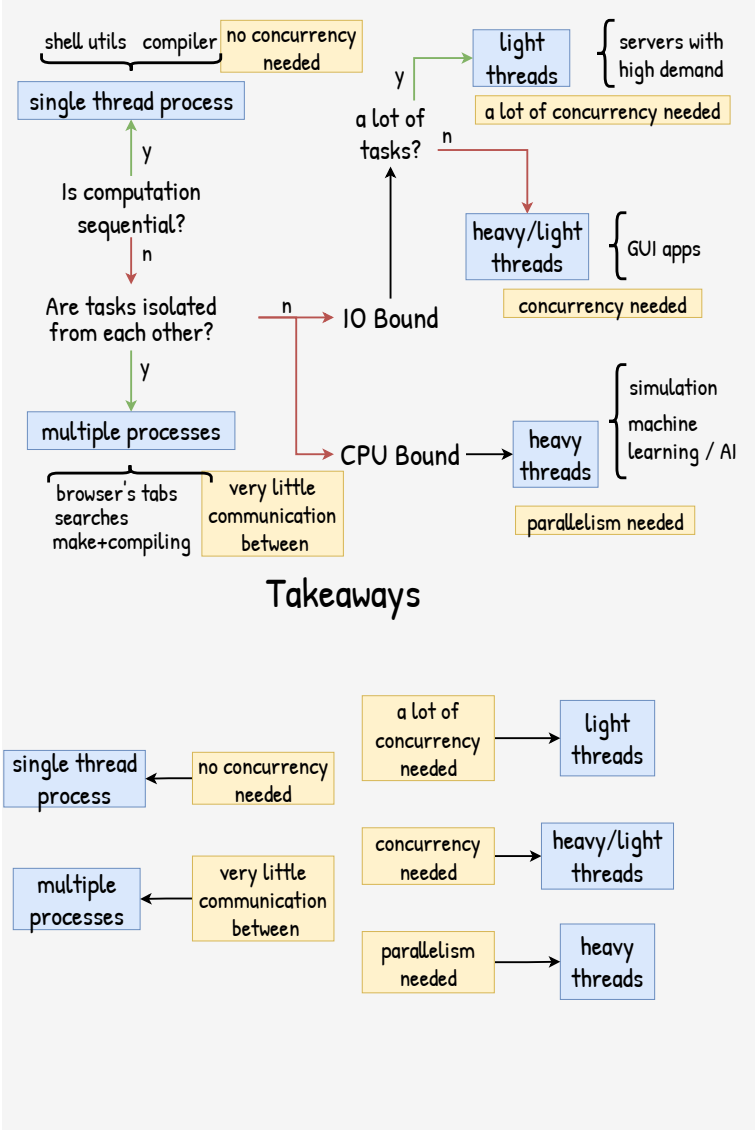
Ejemplos:

- simulaciones
- training de machine learning / AI
- aplicaciones de alta performance

Cuando se necesita concurrencia, los light threads escalan mucho mejor que los heavy threads.

Ejemplos:

- servers con alta demanda (muchos clientes a la vez)




Cuando se necesita concurrencia pero no tanta, tanto los light como los heavy threads funcionan bien.

Ejemplos:

- apps gráficas que requieren estar atentas y procesar los clicks de un mouse

  o el input de un teclado mientras en *"background"* reproducen musica, guardan

  a disco, envian o reciben mensajes por red.

  Quieren tener una UI *"responsive"* sin que las otras tareas bloqueen

  a la UI.

.